

```
(.venv) PS C:\Users\saito\source\repos\AIAgenticEngg\projects\agents\3_crew> crewai create crew  
Multi_Currency_Wallet
```

Creating folder multi_currency_wallet...

Cache expired or not found. Fetching provider data from the web...

Downloading [#####] 1427090/71404

Select a provider to set up:

1. openai
2. anthropic
3. gemini
4. nvidia_nim
5. groq
6. huggingface
7. ollama
8. watson
9. bedrock
10. azure
11. cerebras
12. sambanova
13. other

q. Quit

Enter the number of your choice or 'q' to quit: 1

Select a model to use for Openai:

1. gpt-4
2. gpt-4.1
3. gpt-4.1-mini-2025-04-14
4. gpt-4.1-nano-2025-04-14
5. gpt-4o
6. gpt-4o-mini
7. o1-mini
8. o1-preview

q. Quit

Enter the number of your choice or 'q' to quit: 6

Enter your OPENAI API key (press Enter to skip): sk-proj-xx

API keys and model saved to .env file

Selected model: gpt-4o-mini

- Created multi_currency_wallet\.gitignore
- Created multi_currency_wallet\pyproject.toml
- Created multi_currency_wallet\README.md
- Created multi_currency_wallet\knowledge\user_preference.txt
- Created multi_currency_wallet\src\multi_currency_wallet__init__.py
- Created multi_currency_wallet\src\multi_currency_wallet\main.py
- Created multi_currency_wallet\src\multi_currency_wallet\crew.py
- Created multi_currency_wallet\src\multi_currency_wallet\tools\custom_tool.py
- Created multi_currency_wallet\src\multi_currency_wallet\tools__init__.py
- Created multi_currency_wallet\src\multi_currency_wallet\config\agents.yaml
- Created multi_currency_wallet\src\multi_currency_wallet\config\tasks.yaml

Crew Multi_Currency_Wallet created successfully!

crewai run

```
(.venv) PS C:\Users\saito\source\repos\AIAGenticEngg\projects\agents\3_crew\multi_currency_wallet> crewai run
```

🤖 Agent Started

Agent: Engineering Lead for the engineering team, directing the work of the engineer

Task: Take the high level requirements described here and prepare a detailed design for the engineer; everything should be in 1 python module, but outline the classes and methods in the module. Here are the requirements:

A multi-currency digital wallet system (NomadBank).

Users can hold balances in USD, EUR, GBP, and CAD.

Requirements:

1. Create/Manage account with multiple currency 'pockets'.
2. Deposit USD and Exchange funds between any supported currencies.
3. Record expenses in a specific currency.
4. Prevent spending more than the balance available in a specific pocket.
5. Calculate 'Total Net Worth' in USD using `get_exchange_rate`.
6. Provide a transaction ledger.

IMPORTANT: Only output the design in markdown format, laying out in detail the classes and functions in the module, describing the functionality.

✅ Agent Final Answer

Agent: Engineering Lead for the engineering team, directing the work of the engineer

Final Answer:

```
```markdown
```

```
Design for a Multi-Currency Digital Wallet System (NomadBank)
```

The module ``accounts.py`` will consist of a single class ``Account`` that manages multiple currency pockets, allows deposits, exchanges, expense recording, and provides a transaction ledger with total net worth calculation. The design outlines the class and methods required to fulfill the provided requirements.

```
Module: accounts.py
```

```
Class: Account
```

```
Attributes:
```

- ``balances``: A dictionary to store balances for each currency. Keys are 'USD', 'EUR', 'GBP', 'CAD'.
- ``transaction_ledger``: A list to record all transactions with details.

```
Methods:
```

```
- `__init__(self, initial_usd=0.0) -> None`:
```

- `**Description**`: Initializes the account with a specified initial USD balance and creates empty balances for other currencies.

- `**Parameters**`:

- ``initial_usd`` (float): Initial amount in USD to add to the account (default is 0.0).

```
- `create_pocket(self, currency: str, initial_amount: float = 0.0) -> bool`:
```

- `**Description**`: Creates a currency pocket with an initial amount if it doesn't already exist.

- `**Parameters**`:

- ``currency`` (str): The currency code to create the pocket for (e.g., 'EUR').

- ``initial_amount`` (float): Initial amount in the specified currency (default is 0.0).

- `**Returns**`: ``True`` if pocket was created, ``False`` if it already exists.

```
- `deposit_usd(self, amount: float) -> None`:
```

- `**Description**`: Deposits an amount in USD into the account.

- `**Parameters**`:

- ``amount`` (float): The amount in USD to deposit.

```
- `exchange_currency(self, from_currency: str, to_currency: str, amount: float, exchange_rate: float) -> bool`:
```

- `**Description**`: Exchanges a specified amount from one currency pocket to another using a given exchange rate.

- `**Parameters**`:

```

- `exchange_currency(self, from_currency: str, to_currency: str, amount: float, exchange_rate: float) -> bool`:
- Description: Exchanges a specified amount from one currency pocket to another using a given exchange rate.
- Parameters:
 - `from_currency` (str): The currency to convert from.
 - `to_currency` (str): The currency to convert to.
 - `amount` (float): The amount to exchange.
 - `exchange_rate` (float): The rate at which the `from_currency` will be converted to `to_currency`.
- Returns: `True` if the exchange was successful, `False` if insufficient funds.

- `record_expense(self, currency: str, amount: float) -> bool`:
- Description: Deducts an expense from a specified currency pocket.
- Parameters:
 - `currency` (str): The currency in which to record the expense.
 - `amount` (float): The amount of the expense.
- Returns: `True` if the expense was recorded, `False` if insufficient funds.

- `calculate_net_worth(self, get_exchange_rate: callable) -> float`:
- Description: Calculates total net worth in USD. Utilizes a provided function `get_exchange_rate` to obtain conversion rates.
- Parameters:
 - `get_exchange_rate` (callable): A function that takes two arguments (from_currency, to_currency) and returns the exchange rate.
- Returns: Net worth in USD as a float.

- `add_transaction(self, description: str, amount: float, currency: str) -> None`:
- Description: Adds a transaction to the ledger with a relevant description.
- Parameters:
 - `description` (str): Description of the transaction.
 - `amount` (float): The transaction amount.
 - `currency` (str): The currency of the transaction.

- `get_transaction_ledger(self) -> list`:
- Description: Retrieves the transaction ledger.
- Returns: A list of all transactions recorded.
"""

```

In this design, we have encapsulated all functionalities required by NomadBank in one class, `Account`. The class provides methods to interact with currency pockets, perform operations like deposits and exchanges, record expenses, and maintain a transaction ledger. The `calculate\_net\_worth` function relies on an external function to supply exchange rates, maintaining modularity and adaptability of the code.

#### Task Completion

##### Task Completed

Name: `design_task`

Agent: Engineering Lead for the engineering team, directing the work of the engineer

#### Task Started

##### Task Started

Name: `code_task`

ID: `eb655c07-7640-4f6b-a849-ce8b66f3a1ac`

#### Agent Started

Agent: Python Engineer who can write code to achieve the design described by the engineering lead

Task: Write a python module that implements the design described by the engineering lead, in order to achieve the requirements.

Here are the requirements:

A multi-currency digital wallet system (NomadBank).

Users can hold balances in USD, EUR, GBP, and CAD.

Requirements:

1. Create/Manage account with multiple currency 'pockets'.
2. Deposit USD and Exchange funds between any supported currencies.
3. Record expenses in a specific currency.
4. Prevent spending more than the balance available in a specific pocket.
5. Calculate 'Total Net Worth' in USD using `get_exchange_rate`.
6. Provide a transaction ledger.

Agent: Python Engineer who can write code to achieve the design described by the engineering lead

Final Answer:

```
class Account:
 def __init__(self, initial_usd=0.0) -> None:
 self.balances = {'USD': initial_usd, 'EUR': 0.0, 'GBP': 0.0, 'CAD': 0.0}
 self.transaction_ledger = []

 def create_pocket(self, currency: str, initial_amount: float = 0.0) -> bool:
 if currency not in self.balances:
 return False
 if self.balances[currency] > 0:
 return False
 self.balances[currency] = initial_amount
 return True

 def deposit_usd(self, amount: float) -> None:
 if amount > 0:
 self.balances['USD'] += amount
 self.add_transaction("Deposit", amount, 'USD')

 def exchange_currency(self, from_currency: str, to_currency: str, amount: float, exchange_rate: float) -> bool:
 if self.balances[from_currency] >= amount:
 self.balances[from_currency] -= amount
 amount_converted = amount * exchange_rate
 self.balances[to_currency] += amount_converted
 self.add_transaction(f"Exchange from {from_currency} to {to_currency}", amount, from_currency)
 return True
 return False

 def record_expense(self, currency: str, amount: float) -> bool:
 if self.balances[currency] >= amount:
 self.balances[currency] -= amount
 self.add_transaction("Expense", amount, currency)
 return True
 return False

 def calculate_net_worth(self, get_exchange_rate: callable) -> float:
 total_net_worth = self.balances['USD']
 for currency in ['EUR', 'GBP', 'CAD']:
 exchange_rate = get_exchange_rate(currency, 'USD')
 total_net_worth += self.balances[currency] * exchange_rate
```

```
def calculate_net_worth(self, get_exchange_rate: callable) -> float:
 total_net_worth = self.balances['USD']
 for currency in ['EUR', 'GBP', 'CAD']:
 rate = get_exchange_rate(currency, 'USD')
 total_net_worth += self.balances[currency] * rate
 return total_net_worth

def add_transaction(self, description: str, amount: float, currency: str) -> None:
 self.transaction_ledger.append({'description': description, 'amount': amount, 'currency': currency})

def get_transaction_ledger(self) -> list:
 return self.transaction_ledger
```

Saving report to output/callback\report.txt...

✓ Report successfully saved.

#### 📄 Task Completion

##### Task Completed

Name: code\_task

Agent: Python Engineer who can write code to achieve the design described by the engineering lead

#### 📄 Task Started

##### Task Started

Name: frontend\_task

ID: adaa0c62-99c2-4086-90f4-bdadf9d38f0f

#### 👤 Agent Started

Agent: A Gradio expert to who can write a simple frontend to demonstrate a backend

Task: Write a gradio UI in a module app.py that demonstrates the given backend class in accounts.py. Assume there is only 1 user, and keep the UI very simple indeed - just a prototype or demo. Here are the requirements:

A multi-currency digital wallet system (NomadBank).

Users can hold balances in USD, EUR, GBP, and CAD.

Requirements:

1. Create/Manage account with multiple currency 'pockets'.
2. Deposit USD and Exchange funds between any supported currencies.
3. Record expenses in a specific currency.
4. Prevent spending more than the balance available in a specific pocket.
5. Calculate 'Total Net Worth' in USD using get\_exchange\_rate.
6. Provide a transaction ledger.

Agent: A Gradio expert to who can write a simple frontend to demonstrate a backend

Final Answer:

```
from accounts import Account
import gradio as gr

def get_exchange_rate(currency_from, currency_to):
 rates = {
 'EUR': 1.1, # Example rates
 'GBP': 1.3,
 'CAD': 0.75,
 }
 if currency_to == 'USD':
 return 1 / rates[currency_from] # Convert to USD
 return rates[currency_from] # Example rate to other currencies

account = Account()

def create_pocket(currency, initial_amount):
 success = account.create_pocket(currency, initial_amount)
 return f"Pocket created: {success}"

def deposit_usd(amount):
 account.deposit_usd(amount)
 return f"Deposited: ${amount}"

def exchange_currency(from_currency, to_currency, amount):
 rate = get_exchange_rate(from_currency, to_currency)
 success = account.exchange_currency(from_currency, to_currency, amount, rate)
 return f"Exchange successful: {success}"

def record_expense(currency, amount):
 success = account.record_expense(currency, amount)
 return f"Expense recorded: {success}"
```

```

def record_expense(currency, amount):
 success = account.record_expense(currency, amount)
 return f"Expense recorded: {success}"

def calculate_net_worth():
 net_worth = account.calculate_net_worth(get_exchange_rate)
 return f"Total Net Worth: ${net_worth}"

def get_ledger():
 return account.get_transaction_ledger()

app = gr.Interface(
 fn=lambda currency, amount: create_pocket(currency, amount) if currency else deposit_usd(amount),
 inputs=[
 gr.Dropdown(["USD", "EUR", "GBP", "CAD"], label="Currency for Pocket Creation"),
 gr.Number(label="Initial Amount", default=0.0),
 gr.Number(label="Deposit Amount", default=0.0)
],
 outputs="text",
 title="NomadBank - Multi-Currency Wallet",
 description="Demonstrate the NomadBank Wallet Functionality"
)

app.add_component(gr.Interface(fn=deposit_usd, inputs="number", outputs="text", title="Deposit USD"))
app.add_component(gr.Interface(fn=exchange_currency, inputs=["text", "text", "number"], outputs="text", title="Exchange Currency"))
app.add_component(gr.Interface(fn=record_expense, inputs=["text", "number"], outputs="text", title="Record Expense"))
app.add_component(gr.Interface(fn=calculate_net_worth, inputs=None, outputs="text", title="Calculate Net Worth"))
app.add_component(gr.Interface(fn=get_ledger, inputs=None, outputs="json", title="Transaction Ledger"))

app.launch()

```

#### Task Completion

##### Task Completed

Name: frontend\_task

Agent: A Gradio expert to who can write a simple frontend to demonstrate a backend

#### Execution Traces

 Detailed execution traces are available!

View insights including:

- Agent decision-making process
- Task execution flow and timing
- Tool usage details

Would you like to view your execution traces? [y/N] (20s timeout):

#### Tracing Preference Saved

Info: Tracing has been disabled.

Your preference has been saved. Future Crew/Flow executions will not collect traces.

To enable tracing later, do any one of these:

- Set tracing=True in your Crew/Flow code
- Set CREWAI\_TRACING\_ENABLED=true in your project's .env file
- Run: crewai traces enable

## Crew Completion

Crew Execution Completed

Name: MultiCurrencyWalletCrew

ID: ef049d77-7ce3-43c6-9a61-c1d542fe38e8

Final Output: from accounts import Account

import gradio as gr

```
def get_exchange_rate(currency_from, currency_to):
 rates = {
 'EUR': 1.1, # Example rates
 'GBP': 1.3,
 'CAD': 0.75,
 }
 if currency_to == 'USD':
 return 1 / rates[currency_from] # Convert to USD
 return rates[currency_from] # Example rate to other currencies

account = Account()

def create_pocket(currency, initial_amount):
 success = account.create_pocket(currency, initial_amount)
 return f"Pocket created: {success}"

def deposit_usd(amount):
 account.deposit_usd(amount)
 return f"Deposited: ${amount}"

def exchange_currency(from_currency, to_currency, amount):
 rate = get_exchange_rate(from_currency, to_currency)
 success = account.exchange_currency(from_currency, to_currency, amount, rate)
 return f"Exchange successful: {success}"

def record_expense(currency, amount):
 success = account.record_expense(currency, amount)
 return f"Expense recorded: {success}"

def calculate_net_worth():
 net_worth = account.calculate_net_worth(get_exchange_rate)
 return f"Total Net Worth: ${net_worth}"
```

```
def calculate_net_worth():
 net_worth = account.calculate_net_worth(get_exchange_rate)
 return f"Total Net Worth: ${net_worth}"

def get_ledger():
 return account.get_transaction_ledger()

app = gr.Interface(
 fn=lambda currency, amount: create_pocket(currency, amount) if currency else deposit_usd(amount),
 inputs=[
 gr.Dropdown(["USD", "EUR", "GBP", "CAD"], label="Currency for Pocket Creation"),
 gr.Number(label="Initial Amount", default=0.0),
 gr.Number(label="Deposit Amount", default=0.0)
],
 outputs="text",
 title="NomadBank - Multi-Currency Wallet",
 description="Demonstrate the NomadBank Wallet Functionality"
)

app.add_component(gr.Interface(fn=deposit_usd, inputs="number", outputs="text", title="Deposit USD"))
app.add_component(gr.Interface(fn=exchange_currency, inputs=["text", "text", "number"], outputs="text", title="Exchange Currency"))
app.add_component(gr.Interface(fn=record_expense, inputs=["text", "number"], outputs="text", title="Record Expense"))
app.add_component(gr.Interface(fn=calculate_net_worth, inputs=None, outputs="text", title="Calculate Net Worth"))
app.add_component(gr.Interface(fn=get_ledger, inputs=None, outputs="json", title="Transaction Ledger"))

app.launch()
```

`uv run app.py`

Gradio: <http://127.0.0.1:7860/>

## NomadBank Multi-Currency Wallet

### UI

### NomadBank Multi-Currency Wallet

DepositExchangeExpenseNet WorthLedger

amount

100

output

Deposited: \$100

ClearSubmitFlag

### NomadBank Multi-Currency Wallet

DepositExchangeExpenseNet WorthLedger

from\_currency

USD

to\_currency

CAD

amount

50

output

Exchange successful: False

ClearSubmitFlag

### NomadBank Multi-Currency Wallet

DepositExchangeExpenseNet WorthLedger

currency

CAD

amount

5

output

Expense recorded: True

ClearSubmitFlag

### NomadBank Multi-Currency Wallet

DepositExchangeExpenseNet WorthLedger

output

Total Net Worth: \$94.04999999999998

ClearGenerateFlag

NomadBank Multi-Currency Wallet

Deposit   Exchange   Expense   Net Worth   Ledger

( ) output

```
1 ▼ [
2 ▼ {
3 "description": "Deposit",
4 "amount": 100,
5 "currency": "USD"
6 },
7 ▼ {
8 "description": "Exchange from USD to EUR",
9 "amount": 10,
10 "currency": "USD"
11 },
12 ▼ {
13 "description": "Expense",
14 "amount": 2,
15 "currency": "EUR"
16 },
17 ▼ {
18 "description": "Exchange from USD to CAD",
19 "amount": 50,
20 "currency": "USD"
21 },
22 ▼ {
```

Clear

Generate

Flag